

# SMBA 分组密码算法

王克, 贾文义, 黄念念

兴唐通信科技有限公司, 北京 100191

通信作者: 王克, E-mail: wangke3107@163.com

**摘要:** SMBA 算法是一个安全性高、实用性强、创新设计的分组密码算法. SMBA 算法支持三种版本, 当分组长度为 128 比特时, 密钥长度支持 128 比特和 256 比特, 当分组长度为 256 比特时, 密钥长度支持 256 比特. SMBA 算法采用经典的 Feistel 结构, 加解密过程一致, 轮函数采用 SP 结构, 密钥扩展算法使用广义 Feistel 结构. 算法创新设计了非线性部件  $S_0$  和  $S_1$ , 以及线性部件  $L_{64}$ , 均具有良好的密码学性质.  $S_0$  和  $S_1$  均由代数构造生成, 具有较优的差分概率、线性概率和代数次数, 且乘法复杂度低, 侧信道防护代价小.  $L_{64}$  由简单线性变换组合构成, 具有较高的差分 and 线性分支数, 关键路径短, 延时低. SMBA 算法在不同版本中共用主要部件  $S_0$ 、 $S_1$  和  $L_{64}$ , 扩展自然. SMBA 算法的安全界清晰, 可以有效评估抵抗差分攻击、线性攻击、积分攻击等已知攻击的强度, 有较大的安全冗余. 算法的基本运算为查表、循环移位以及异或等逻辑运算, 易于软件和硬件实现. 算法实现方式灵活, 软硬件速度快, 可适用于多种平台 (如 8/32/64 位平台).

**关键词:** SMBA 算法; S 盒; 线性变换; 差分攻击; 线性攻击

**中图分类号:** TP309.7      **文献标识码:** A      **DOI:** 10.13868/j.cnki.jcr.000341

中文引用格式: 王克, 贾文义, 黄念念. SMBA 分组密码算法[J]. 密码学报, 2019, 6(6): 786–802.

英文引用格式: WANG K, JIA W Y, HUANG N N. On the SMBA block cipher[J]. Journal of Cryptologic Research, 2019, 6(6): 786–802.

## On the SMBA Block Cipher

WANG Ke, JIA Wen-Yi, HUANG Nian-Nian

Xingtang Telecommunication Technology Co. Ltd., Beijing 100191, China

Corresponding author: WANG Ke, E-mail: wangke3107@163.com

**Abstract:** SMBA is a block cipher with high security, good flexibility, and novel design. SMBA supports three versions, which are 128-bit block size with 128-bit and 256-bit keys, and 256-bit block size with 256-bit keys. SMBA uses the classical Feistel structure which has the same encryption and decryption procedure, the round function uses the SP network, and the key schedule uses a generalized Feistel structure. Two nonlinear transformations  $S_0$  and  $S_1$ , and a linear transformation  $L_{64}$ , are novelly designed, all of which have good cryptographic properties.  $S_0$  and  $S_1$  are algebraically constructed, with good differential probability, linear bias and algebraic degree, and both of them have lower multiplication complexity which leads to low cost on side-channel resistance.  $L_{64}$  is constructed by simple linear transformations, and it has large differential and linear branch number and short

implementation path. SMBA has a natural expansion by commonly using  $S_0$ ,  $S_1$ , and  $L_{64}$  in different versions. SMBA has a clear security margin with enough redundancy, and it can efficiently evaluate the ability to resist many known attacks, such as differential attacks, linear attacks, and integral attacks. The basic operations in SMBA are table lookups, rotations, and XOR logical operations, which is suitable for software and hardware implementation. SMBA has flexible implementations, high speeds and suitable for implementations on various platforms (8/32/64-bit platforms).

**Key words:** SMBA; S-box; linear transformation; differential attack; linear attack

1 算法描述

1.1 概述

SMBA 是一个分组密码算法, 分组长度支持 128 和 256 比特. 分组长度为 128 比特时密钥长度支持 128 和 256 比特, 分组长度为 256 比特时密钥长度为 256 比特. 分组长度 (比特数, 记为 BL) 和密钥长度 (比特数, 记为 KL) 对应的 SMBA 的迭代轮数 (记为  $N$ ) 如表 1 所示.

表 1 SMBA 算法的三个版本和迭代轮数  
Table 1 Three versions of SMBA and the iterated round number

| 分组长度   | 密钥长度   |        |
|--------|--------|--------|
|        | 128 比特 | 256 比特 |
| 128 比特 | 18     | 24     |
| 256 比特 | —      | 24     |

分组长度为 128 比特的 SMBA 记为 SMBA-128, 其中密钥长度为 128 或 256 比特时也分别记为 SMBA-128-128 或 SMBA-128-256. 分组长度为 256 比特的 SMBA 记为 SMBA-256, 也可记为 SMBA-256-256.

1.2 符号、运算符和约定

1.2.1 符号和运算符列表

|             |                                   |          |                   |
|-------------|-----------------------------------|----------|-------------------|
| 0x          | 用前缀 0x 表示 16 进制数                  | =        | 赋值运算符, 也可表示相等     |
| $M^T$       | 向量或矩阵 $M$ 的转置                     | $\oplus$ | 逐比特异或运算符          |
| $\parallel$ | 字符串连接运算符                          | $\lll n$ | 操作数循环左移 $n$ 比特运算符 |
| $0^n$       | $n$ 比特 0 组成的比特串                   | BL       | 分组长度, 比特数.        |
| KL          | 密钥长度, 比特数                         | $N$      | 迭代轮数              |
| $MSB_t(x)$  | 由 $x$ 的最左 (即最高) $t$ 比特组成的 $t$ 比特数 |          |                   |
| $LSB_t(x)$  | 由 $x$ 的最右 (即最低) $t$ 比特组成的 $t$ 比特数 |          |                   |

1.2.2 比特、字节和子块顺序约定

所有数据变量用最高比特 (或子块) 在左边, 最低比特 (或子块) 在右边表示, 而且从左到右从 0 开始编号. 例如, 一个  $n$  比特数从左到右分别称为它的第 0 比特、第 1 比特、第 2 比特、 $\cdots$ 、第  $n-1$  比特. 当一个变量被分成几个子块时, 最左边 (最高) 子块由原始数据的最高部分组成, 等等一直到最低.

当密码算法的输入 (或输出) 是字节流时, 我们按地址小的是高位 (即左边), 地址大的是低位 (即右边) 的顺序把每 4 (或 2) 个字节组成一个 32 (或 16) 比特字. 32 (或 16) 比特字之间的顺序是先左后右, 即存储时, 左边地址小, 右边地址大.

约定比特串  $x_0\parallel x_1\parallel x_2\parallel \cdots \parallel x_{n-1}$ 、整数  $2^{n-1}x_0 + 2^{n-2}x_1 + 2^{n-3}x_2 + \cdots + 2x_{n-2} + x_{n-1}$ 、行向量  $(x_0, x_1, x_2, \cdots, x_{n-1})$ 、列向量  $(x_0, x_1, x_2, \cdots, x_{n-1})^T$  是等价的, 可互相转化.

### 1.3 加解密过程

#### 1.3.1 加密过程

##### 算法 1 SMBA 加密算法

1. 将 BL 比特明文 plaintext 分为各 BL/2 比特的左、右两部分  $x_0$  和  $x_1$ , 即  $\text{plaintext} = x_0 \| x_1$
2. 对于  $i = 0, 1, \dots, N-1$ 

$$x_{i+2} = x_i \oplus F[\text{EK}_i](x_{i+1})$$
3.  $x_{N+1}$  和  $x_N$  连接成 BL 比特密文 ciphertext, 即  $\text{ciphertext} = x_{N+1} \| x_N$ .

其中,  $F$  为轮函数,  $\text{EK}_i (0 \leq i \leq N-1)$  为  $N$  个 BL/2 比特的加密子密钥, 由密钥 Key 用密钥扩展算法派生.

#### 1.3.2 解密过程

##### 算法 2 SMBA 解密算法

1. 将 BL 比特密文 ciphertext 分为各 BL/2 比特的左、右两部分  $x_0$  和  $x_1$ , 即  $\text{ciphertext} = x_0 \| x_1$
2. 对于  $i = 0, 1, \dots, N-1$ 

$$x_{i+2} = x_i \oplus F[\text{DK}_i](x_{i+1})$$
3.  $x_{N+1}$  和  $x_N$  连接成 BL 比特明文 plaintext, 即  $\text{plaintext} = x_{N+1} \| x_N$ .

其中,  $F$  为轮函数,  $\text{DK}_i (0 \leq i \leq N-1)$  为  $N$  个 BL/2 比特的解密子密钥, 由密钥 Key 用密钥扩展算法派生.

解密子密钥与加密子密钥满足如下关系:  $\text{DK}_i = \text{EK}_{N-1-i} (0 \leq i \leq N-1)$ .

### 1.4 轮函数及其组件

#### 1.4.1 轮函数 $F[K]$

轮函数  $y = F[K](x)$  的输入是一个 BL/2 比特数  $x$  和一个 BL/2 比特子密钥  $K$ , 输出是一个 BL/2 比特数  $y$ ,  $y = F[K](x) = L(S(P(W[K](x))))$ , 即  $F[K] = L \circ S \circ P \circ W[K]$  (参见图1、图2). 各变换  $W[K]$ 、 $P$ 、 $S$ 、 $L$  见下面定义.

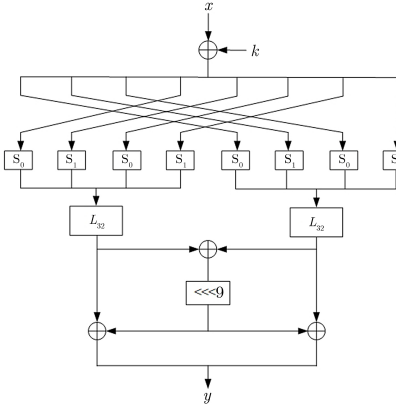


图 1 SMBA-128 轮函数

Figure 1 Round function of SMBA-128

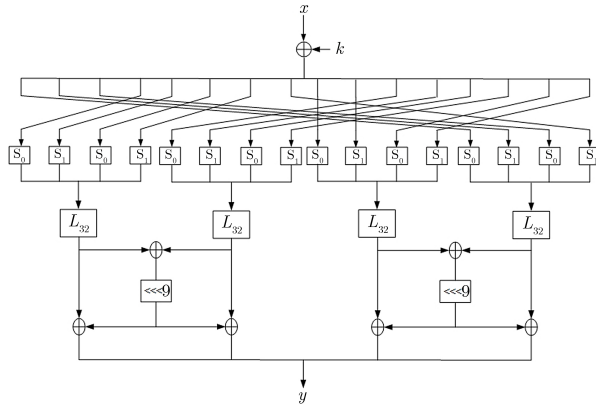


图 2 SMBA-256 轮函数

Figure 2 Round function of SMBA-256

#### 1.4.2 白化变换 $W[K]$

对于 BL/2 比特数  $x$  和 BL/2 比特子密钥  $K$ ,  $W[K](x) = x \oplus K$ .

#### 1.4.3 换位变换 $P$

换位变换  $P$  对于 SMBA-128 使用换位变换  $P_{64}$ , 对于 SMBA-256 使用换位变换  $P_{128}$ .

$P_{64}$  是一个 64 比特数到 64 比特数的变换, 具体为: 把 64 比特数  $x$  分为 8 个 8 比特数  $x_i$ ,  $0 \leq i \leq 7$ , 即  $x = x_0 \| x_1 \| \dots \| x_7$ , 令  $y_i = x_{7-i}$ , 则

$$P_{64}(x) = y_0 \| y_1 \| \dots \| y_7$$

其中  $\tau(i)$ ,  $0 \leq i \leq 7$ , 依次为: 3, 4, 5, 6, 0, 1, 2, 7.

$P_{128}$  是一个 128 比特数到 128 比特数的变换, 具体为: 把 128 比特数  $x$  分为 16 个 8 比特数  $x_i$ ,  $0 \leq i \leq 15$ , 即  $x = x_0 \| x_1 \| \cdots \| x_{15}$ , 令  $y_i = x_{\pi(i)}$ , 则

$$P_{128}(x) = y_0 \| y_1 \| \cdots \| y_{15}$$

其中  $\pi(i)$ ,  $0 \leq i \leq 15$ , 依次为: 3, 4, 5, 6, 10, 11, 12, 13, 8, 9, 14, 15, 0, 1, 2, 7.

#### 1.4.4 代替变换 $S$

代替变换  $S$  对于 SMBA-128 使用代替变换  $S_{64}$ , 对于 SMBA-256 使用代替变换  $S_{128}$ .

$S_{32}$  是一个 32 比特数到 32 比特数的变换, 具体为: 将 32 比特数  $x$  分为 4 个 8 比特数, 即  $x = x_0 \| x_1 \| x_2 \| x_3$ , 令

$$y_0 = S_0(x_0), y_1 = S_1(x_1), y_2 = S_0(x_2), y_3 = S_1(x_3)$$

则  $S_{32}(x) = y_0 \| y_1 \| y_2 \| y_3$ , 其中,  $S_0$ 、 $S_1$  都是 8 比特到 8 比特的代替盒 (S 盒).

$S_{64}$  是一个 64 比特数到 64 比特数的变换, 具体为: 将 64 比特数  $x$  分为 2 个 32 比特数, 即  $x = x_0 \| x_1$ , 令

$$y_0 = S_{32}(x_0), y_1 = S_{32}(x_1)$$

则  $S_{64}(x) = y_0 \| y_1$ .

$S_{128}$  是一个 128 比特数到 128 比特数的变换, 具体为: 将 128 比特数  $x$  分为 2 个 64 比特数, 即  $x = x_0 \| x_1$ , 令

$$y_0 = S_{64}(x_0), y_1 = S_{64}(x_1)$$

则  $S_{128}(x) = y_0 \| y_1$ .

#### 1.4.5 线性变换 $L$

线性变换  $L$  对于 SMBA-128 使用线性变换  $L_{64}$ , 对于 SMBA-256 使用线性变换  $L_{128}$ .

$L_{32}$  是 32 比特数到 32 比特数的变换, 具体为: 将 32 比特数  $x$  分为 4 个 8 比特数, 即  $x = x_0 \| x_1 \| x_2 \| x_3$ , 令

$$y_0 = x_0 \oplus x_2 \oplus x_3$$

$$y_1 = x_1 \oplus x_2 \oplus x_3$$

$$y_2 = x_0 \oplus x_1 \oplus x_2$$

$$y_3 = x_0 \oplus x_1 \oplus x_3$$

则  $L_{32}(x) = y_0 \| y_1 \| y_2 \| y_3$ .

$L_{64}$  是一个 64 比特数到 64 比特数的变换, 具体为: 将 64 比特数  $x$  分为 2 个 32 比特数, 即  $x = x_0 \| x_1$ , 令

$$z_0 = L_{32}(x_0), z_1 = L_{32}(x_1)$$

$$u = z_0 \oplus z_1$$

$$v = u \lll 9$$

$$y_0 = z_0 \oplus v, y_1 = z_1 \oplus v$$

则  $L_{64}(x) = y_0 \| y_1$ .

$L_{128}$  是一个 128 比特数到 128 比特数的变换, 具体为: 将 128 比特数  $x$  分为 2 个 64 比特数, 即  $x = x_0 \| x_1$ , 令

$$y_0 = L_{64}(x_0), y_1 = L_{64}(x_1)$$

则  $L_{128}(x) = y_0 \| y_1$ .

### 1.5 密钥扩展

密钥扩展算法根据密钥 Key, 生成  $N$  个 BL/2 比特加密子密钥  $EK_i$  和  $N$  个 BL/2 比特解密子密钥  $DK_i$ ,  $0 \leq i \leq N-1$ .

常数  $C$  为自然对数的底  $e$  的小数部分的前 128 比特, 用 16 进制表示为

$$C = 0xb7e15162, 0x8aed2a6a, 0xbf715880, 0x9cf4f3c7$$

#### 1.5.1 变换 $\alpha$ 、 $\beta$ 、 $\gamma$

变换  $\alpha$ 、 $\beta$ 、 $\gamma$  对于 SMBA-128-128 和 SMBA-128-256 使用  $\alpha_{64}$ 、 $\beta_{64}$ 、 $\gamma_{64}$ , 对于 SMBA-256-256 使用  $\alpha_{128}$ 、 $\beta_{128}$ 、 $\gamma_{128}$ .

$\alpha_{64}$ 、 $\beta_{64}$ 、 $\gamma_{64}$  都是 64 比特到 64 比特的变换, 分别如下.

把 64 比特数  $x$  分为 8 个 8 比特数  $x_i$ ,  $0 \leq i \leq 7$ , 即  $x = x_0 \| x_1 \| \cdots \| x_7$ , 令  $y_i = x_{\alpha_{64}(i)}$ , 则

$$\alpha_{64}(x) = y_0 \| y_1 \| \cdots \| y_7$$

其中  $\alpha_{64}(i)$ ,  $0 \leq i \leq 7$ , 依次为: 2, 7, 1, 4, 6, 3, 5, 0.

$$\beta_{64}(x) = x \lll 16$$

把 64 比特数  $x$  分为 8 个 8 比特数  $x_i$ ,  $0 \leq i \leq 7$ , 即  $x = x_0 \| x_1 \| \cdots \| x_7$ , 令  $y_i = S_{i \bmod 2}(x_i)$ , 则

$$\gamma_{64}(x) = y_0 \| y_1 \| \cdots \| y_7$$

$\alpha_{128}$ 、 $\beta_{128}$ 、 $\gamma_{128}$  都是 128 比特到 128 比特的变换, 分别如下.

把 128 比特数  $x$  分为 16 个 8 比特数  $x_i$ ,  $0 \leq i \leq 15$ , 即  $x = x_0 \| x_1 \| \cdots \| x_{15}$ , 令  $y_i = x_{\alpha_{128}(i)}$ , 则

$$\alpha_{128}(x) = y_0 \| y_1 \| \cdots \| y_{15}$$

其中  $\alpha_{128}(i)$ ,  $0 \leq i \leq 15$ , 依次为: 13, 4, 12, 0, 8, 10, 2, 6, 14, 3, 11, 15, 7, 9, 1, 5.

$$\beta_{128}(x) = x \lll 32$$

把 128 比特数  $x$  分为 16 个 8 比特数  $x_i$ ,  $0 \leq i \leq 15$ , 即  $x = x_0 \| x_1 \| \cdots \| x_{15}$ , 令  $y_i = S_{i \bmod 2}(x_i)$ , 则

$$\gamma_{128}(x) = y_0 \| y_1 \| \cdots \| y_{15}$$

#### 1.5.2 SMBA-128-128 的子密钥生成

---

##### 算法 3 SMBA-128-128 子密钥生成算法

---

1.  $U_0 = \text{MSB}_{64}(\text{Key})$ ,  $V_0 = \text{LSB}_{64}(\text{Key})$
  2.  $D = N \| \text{BlockLen} \| \text{KeyLen} \| 0^{40}$ , 这里  $\text{BlockLen} = \text{BL}/8$  为分组长度 (字节数),  $\text{KeyLen} = \text{KL}/8$  为密钥长度 (字节数),  $N$ 、 $\text{BlockLen}$ 、 $\text{KeyLen}$  都用 8 比特表示
  3.  $C_0 = \text{MSB}_{64}(C) \oplus D$
  4. 进行  $N$  个密钥扩展轮变换 (参见图 3), 即  
 对于  $i = 0$  到  $N-1$ 

$$\begin{aligned} X_i &= U_i \oplus C_i \\ Y_i &= \alpha_{64}(X_i) \\ Z_i &= \gamma_{64}(Y_i) \\ EK_i &= DK_{N-1-i} = Z_i \\ U_{i+1} &= Z_i \oplus V_i \\ V_{i+1} &= \beta_{64}(U_i) \\ C_{i+1} &= C_i \lll 23 \end{aligned}$$
-

## 1.5.3 SMBA-128-256 的子密钥生成

## 算法 4 SMBA-128-256 子密钥生成算法

1.  $U_{0,0}||U_{0,1}||U_{0,2}||U_{0,3} = \text{Key}$ , 其中  $U_{0,i}$  的长度都是 64 比特,  $0 \leq i \leq 3$ ;
2.  $D = N||\text{BlockLen}||\text{KeyLen}||0^{40}$ , 这里  $\text{BlockLen} = \text{BL}/8$  为分组长度 (字节数),  $\text{KeyLen} = \text{KL}/8$  为密钥长度 (字节数),  $N$ 、 $\text{BlockLen}$ 、 $\text{KeyLen}$  都用 8 比特表示
3.  $C_0 = \text{MSB}_{64}(C) \oplus D$
4. 进行  $N$  个密钥扩展轮变换 (参见图 4), 即  
对于  $i = 0$  到  $N - 1$   
 $X_i = U_{i,0} \oplus C_i$   
 $Y_i = \alpha_{64}(X_i)$   
 $Z_i = \gamma_{64}(Y_i)$   
 $\text{EK}_i = \text{DK}_{N-1-i} = Z_i$   
 $U_{i+1,0} = Z_i \oplus U_{i,1}$   
 $U_{i+1,1} = U_{i,2}$   
 $U_{i+1,2} = U_{i,3}$   
 $U_{i+1,3} = \beta_{64}(U_{i,0})$   
 $C_{i+1} = C_i \lll 23$

## 1.5.4 SMBA-256-256 的子密钥生成

## 算法 5 SMBA-256-256 子密钥生成算法

1.  $U_0 = \text{MSB}_{128}(\text{Key})$ ,  $V_0 = \text{LSB}_{128}(\text{Key})$
2.  $D = N||\text{BlockLen}||\text{KeyLen}||0^{104}$ , 这里  $\text{BlockLen} = \text{BL}/8$  为分组长度 (字节数),  $\text{KeyLen} = \text{KL}/8$  为密钥长度 (字节数),  $N$ 、 $\text{BlockLen}$ 、 $\text{KeyLen}$  都用 8 比特表示
3.  $C_0 = C \oplus D$
4. 进行  $N$  个密钥扩展轮变换 (参见图 3), 即  
对于  $i = 0$  到  $N - 1$   
 $X_i = U_i \oplus C_i$   
 $Y_i = \alpha_{128}(X_i)$   
 $Z_i = \gamma_{128}(Y_i)$   
 $\text{EK}_i = \text{DK}_{N-1-i} = Z_i$   
 $U_{i+1} = Z_i \oplus V_i$   
 $V_{i+1} = \beta_{128}(U_i)$   
 $C_{i+1} = C_i \lll 57$

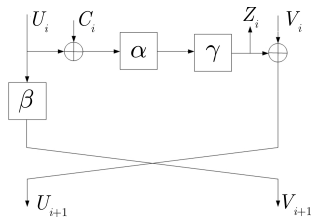


图 3 SMBA-128-128 和 SMBA-256-256 密钥扩展过程  
Figure 3 Key schedule of SMBA-128-128 and SMBA-256-256

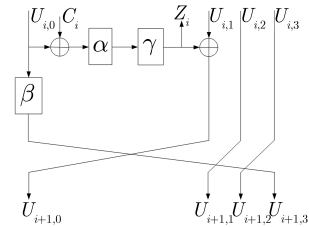


图 4 SMBA-128-256 密钥扩展过程  
Figure 4 Key schedule of SMBA-128-256

1.6 S 盒  $S_0$  和  $S_1$ 

用 16 进制表示的代替盒  $S_0$  和  $S_1$  分别如下:

$S_0$ :

0x18, 0x32, 0x6b, 0x69, 0x9e, 0xcb, 0x0b, 0x2f, 0x03, 0x94, 0x9f, 0xc8, 0x86, 0x26, 0xf4,  
 0x46, 0x67, 0x0d, 0x6d, 0x39, 0xc0, 0x09, 0x29, 0xa1, 0xda, 0x82, 0xf3, 0x78, 0x6a, 0xee, 0x3b,  
 0x84, 0x87, 0xd1, 0xf8, 0x70, 0xb9, 0xa3, 0x3f, 0xfe, 0x3d, 0x37, 0xf7, 0xfb, 0x48, 0xc6, 0xbc,

0x65,0x4e,0x6e,0x64,0xaf,0x91,0xdb,0x1c,0x71,0x8e,0x59,0x8d,0x01,0x74,0xe3,0xa8,  
 0xb4,0xbf,0x4b,0x2d,0xea,0x95,0x6f,0x2b,0x5f,0xb3,0x98,0x57,0xfd,0x24,0x85,0xe9,  
 0x5d,0x6c,0x4d,0x4f,0x13,0xe1,0x49,0x0f,0xdc,0x36,0x55,0x3c,0xef,0x11,0x50,0x4a,  
 0x73,0x5b,0x8a,0xa9,0x92,0x58,0x35,0x80,0x38,0x04,0x06,0x12,0xb7,0xc7,0x68,0xd2,  
 0x45,0x66,0xcd,0x47,0xac,0xba,0xf2,0x16,0x4c,0xb2,0x5c,0x99,0xc2,0x8b,0x23,0x7c,  
 0xe5,0x9b,0xab,0x42,0xad,0x05,0x28,0xce,0xca,0x5e,0x79,0x2e,0xf5,0x72,0x43,0x77,  
 0x96,0x33,0x7a,0xa4,0x40,0xde,0x75,0xf9,0x89,0xd9,0xb5,0x1a,0x9a,0xb6,0xd6,0x8f,  
 0xb0,0xe4,0x62,0x88,0x7e,0x83,0x15,0x3a,0x2c,0x51,0x56,0xeb,0x90,0xf1,0x1b,0x00,  
 0x9d,0x19,0x52,0xa7,0x34,0x53,0x97,0x25,0xbe,0x30,0xaa,0xf0,0xa2,0x02,0x0c,0xe7,  
 0xd3,0x08,0xc1,0xf6,0xa0,0x76,0x10,0x1f,0x61,0xa5,0x14,0xec,0x22,0x31,0xd8,0xbb,  
 0xc5,0xe8,0x54,0x93,0x3e,0x7b,0x9c,0x27,0xdd,0x1d,0xcc,0xe2,0x0e,0xe6,0x81,0x20,  
 0x2a,0xfa,0xff,0xe0,0x07,0xae,0xcf,0x0a,0xed,0x1e,0xa6,0x41,0x7f,0x63,0x5a,0xfc,  
 0xd5,0xc9,0xd4,0xb8,0x60,0xd7,0x7d,0xd0,0xc4,0xc3,0x44,0x17,0xdf,0x8c,0xb1,0xbd  
 $S_1$ :

0x6a,0x05,0xe8,0x16,0x7f,0x37,0x68,0x54,0xbe,0x01,0xc1,0x87,0xbf,0xb3,0xd1,0x44,  
 0xf8,0xf9,0xd4,0xf5,0x1f,0x77,0xa5,0x3b,0x04,0x92,0x62,0xd7,0xc6,0xa7,0x0c,0xb1,  
 0xbd,0x5c,0xff,0xf4,0x17,0x3d,0xa3,0x32,0x9e,0x09,0x8c,0x63,0x66,0x10,0xe9,0x47,  
 0xa8,0xfe,0x3c,0xc0,0x7d,0x1d,0xfd,0x4d,0x4c,0xd8,0xd9,0x9c,0x34,0xd0,0x25,0xeb,  
 0x91,0x61,0xe0,0x1b,0x95,0x98,0xe5,0xba,0xe3,0x64,0xad,0x0f,0x73,0x19,0x39,0xbb,  
 0x00,0xe7,0xf2,0x3a,0x43,0x79,0x8d,0xf0,0x78,0x23,0xc4,0xa0,0x0b,0x03,0xb4,0x52,  
 0x69,0xf1,0xf6,0xc9,0x0d,0xb6,0x8a,0xa9,0x27,0xf3,0x86,0x88,0xea,0xc2,0x29,0x14,  
 0x8f,0x22,0x9b,0xc7,0x26,0xed,0xa2,0xab,0xd5,0xaa,0xc5,0x6d,0x60,0x42,0x65,0xa6,  
 0x99,0xb2,0xac,0x81,0x3f,0x5d,0xa1,0x72,0x74,0x30,0xc3,0xef,0x9a,0x1e,0x7b,0x2a,  
 0x67,0x48,0x24,0x31,0x35,0x15,0xf7,0x2f,0x89,0x20,0x40,0xdd,0x50,0x2e,0x08,0x11,  
 0xde,0x80,0x5b,0x6c,0x5f,0x75,0x4a,0x7e,0x5e,0x70,0xce,0x6b,0xcf,0x84,0xda,0xec,  
 0x76,0xb7,0x83,0xee,0x55,0x57,0xcd,0x51,0x8b,0x02,0xb9,0xbc,0xd3,0xb5,0xae,0x90,  
 0x46,0xe2,0x71,0xc8,0x6e,0x85,0x8e,0x36,0x38,0x2b,0x18,0x0a,0xb0,0xfa,0x3e,0x21,  
 0x59,0x7c,0x7a,0xb8,0x6f,0x96,0xcc,0x1c,0xe1,0x2c,0x06,0xfc,0x97,0x9d,0xe4,0x07,  
 0xca,0x0e,0xcb,0xd6,0x4b,0x13,0x4f,0x49,0xdf,0x82,0x4e,0xdc,0x5a,0x1a,0xdb,0x53,  
 0x56,0x33,0xaf,0xfb,0x9f,0xd2,0x2d,0x41,0x45,0x93,0x58,0x28,0xe6,0xa4,0x12,0x94

#### 1.6.1 $S_0$ 构造过程

设  $Z$  是  $\text{GF}(2)$  中的不可约多项式  $z^2 + z + 1$  的根, 则  $(Z, 1)$  是  $\text{GF}(2^2)$  在  $\text{GF}(2)$  上的一组多项式基.

设  $Y$  是  $\text{GF}(2^2)$  中的不可约多项式  $y^2 + y + Z$  (因为  $\text{tr}_{4/2}(Z) = Z + Z^2 = 1$ ) 的根, 则  $(Y, 1)$  是  $\text{GF}(2^4)$  在  $\text{GF}(2^2)$  上的一组多项式基,  $(ZY, Y, Z, 1)$  是  $\text{GF}(2^4)$  在  $\text{GF}(2)$  上的一组基. 有

$$z^2 + z + 1 = (z + Z)(z + Z^2), y^2 + y + Z = (y + Y)(y + Y^4)$$

$\text{GF}(2^2)$  在基  $(Z, 1)$  上的乘法:

$$\begin{aligned} (b_0Z + b_1)(c_0Z + c_1) &= (b_0c_0 + b_0c_1 + b_1c_0)Z + (b_0c_0 + b_1c_1) \\ &= ((b_0 + b_1)(c_0 + c_1) + b_1c_1)Z + (b_0c_0 + b_1c_1) \end{aligned}$$

$\text{GF}(2^2)$  在基  $(Z, 1)$  上的乘法:

$$\begin{aligned} Z(b_0Z + b_1)(c_0Z + c_1) &= ((b_0c_1 + b_1c_0) + b_1c_1)Z + (b_0c_0 + (b_0c_1 + b_1c_0)) \\ &= ((b_0 + b_1)(c_0 + c_1) + b_0c_0)Z + ((b_0 + b_1)(c_0 + c_1) + b_1c_1) \end{aligned}$$

$\text{GF}(2^4)$  在基  $(Y, 1)$  上的乘法:

$$(b_0Y + b_1)(c_0Y + c_1) = ((b_0 + b_1)(c_0 + c_1) + b_1c_1)Y + (Zb_0c_0 + b_1c_1)$$

设  $S_0$  的 8 比特输入为  $x$ , 则如算法 6 计算  $S_0$  的 8 比特输出  $S_0[x]$ :

算法 6  $S_0$  构造算法 (见图 5)

1.  $a = M_0 \times (x \oplus \text{Con}_0)$ , 将  $a$  分为 2 个 4 比特  $a_0$ 、 $a_1$ , 即  $a = a_0 \| a_1$
2.  $b = a_0 \otimes a_1$
3.  $c = b^{13}$
4.  $d = c \otimes a_1$
5.  $e_0 = Q_0[a_0]$ ,  $e_1 = Q_1[a_1]$
6. 若  $c$  为 0, 则  $f = e_0 \| e_1$ ; 否则  $f = d \| c$
7.  $S_0[x] = M_1 \times (f) \oplus \text{Con}_1$

其中, 符号  $\otimes$  表示  $\text{GF}(2^4)$  在基  $(ZY, Y, Z, 1)$  上的乘法,  $\times$  表示矩阵与列向量相乘.  $b^{13}$  表示  $\text{GF}(2^4)$  在基  $(ZY, Y, Z, 1)$  上  $b$  的 13 次幂, 该变换用 16 进制表示为:

$$0x0, 0x1, 0x2, 0x3, 0x9, 0xb, 0xc, 0xf, 0xe, 0x5, 0xd, 0x4, 0x7, 0x8, 0xa, 0x6$$

两个 4 比特 S 盒  $Q_0$ 、 $Q_1$  分别为:

$$0x0, 0x8, 0x4, 0x2, 0xa, 0xd, 0x9, 0x1, 0xb, 0x6, 0xf, 0xe, 0x5, 0xc, 0x3, 0x7$$

$$0x0, 0x2, 0x8, 0x1, 0x6, 0x9, 0x7, 0x5, 0x4, 0xa, 0xc, 0xd, 0x3, 0xf, 0xe, 0xb$$

两个矩阵  $M_0$  和  $M_1$  都是置换矩阵,  $M_0$  的第  $i$  行第  $\tau_0(i)$  列为 1, 其余为 0;  $M_1$  的第  $i$  行第  $\tau_1(i)$  列为 1, 其余为 0;  $0 \leq i \leq 7$ .  $\tau_0$  和  $\tau_1$  分别为:

$$0, 4, 2, 6, 5, 3, 1, 7$$

$$1, 4, 7, 0, 2, 5, 6, 3$$

两个 8 比特常数分别为:  $\text{Con}_0 = 0x52$ ,  $\text{Con}_1 = 0x4d$ .

### 1.6.2 $S_1$ 构造过程

当  $q = 2^n$  时,  $(\alpha, \alpha^q)$  是  $\text{GF}(q^2)$  在  $\text{GF}(q)$  上的一组正规基, 当且仅当  $\text{tr}_{q^2/q}(\alpha) = \alpha + \alpha^q \neq 0$ . 设  $Z$  是  $\text{GF}(2)$  中的不可约多项式  $z^2 + z + 1$  的根, 则  $(Z^2, Z)$  是  $\text{GF}(2^2)$  在  $\text{GF}(2)$  上的一组正规基.

设  $Y$  是  $\text{GF}(2^2)$  中的不可约多项式  $y^2 + y + Z$  (因为  $\text{tr}_{4/2}(Z) = 1$ ) 的根, 则  $(Y^4, Y)$  是  $\text{GF}(2^4)$  在  $\text{GF}(2^2)$  上的一组正规基,  $(Z^2Y^4, ZY^4, Z^2Y, ZY)$  是  $\text{GF}(2^4)$  在  $\text{GF}(2)$  上的一组基.

设  $X$  是  $\text{GF}(2^4)$  中的不可约多项式  $x^2 + x + Z^2Y^4$  (因为  $\text{tr}_{16/4}(Z^2Y^4) = 1$ ) 的根, 则  $(X^{16}, X)$  是  $\text{GF}(2^8)$  在  $\text{GF}(2^4)$  上的一组正规基. 有  $z^2 + z + 1 = (z + Z)(z + Z^2)$ ,  $y^2 + y + Z = (y + Y)(y + Y^4)$ ,  $x^2 + x + Z^2Y^4 = (x + X)(x + X^{16})$ . 约定  $0^{-1} = 0$ .

$\text{GF}(2^2)$  在基  $(Z^2, Z)$  上的乘法:

$$(b_0Z^2 + b_1Z)(c_0Z^2 + c_1Z) = ((b_0 + b_1)(c_0 + c_1) + b_0c_0)Z^2 + ((b_0 + b_1)(c_0 + c_1) + b_1c_1)Z$$

$\text{GF}(2^2)$  在基  $(Z^2, Z)$  上的乘法:

$$Z(b_0Z^2 + b_1Z)(c_0Z^2 + c_1Z) = (b_0c_0 + b_1c_1)Z^2 + ((b_0 + b_1)(c_0 + c_1) + b_0c_0)Z$$

$\text{GF}(2^4)$  在基  $(Y^4, Y)$  上的乘法:

$$(b_0Y^4 + b_1Y)(c_0Y^4 + c_1Y) = (Z(b_0 + b_1)(c_0 + c_1) + b_0c_0)Y^4 + (Z(b_0 + b_1)(c_0 + c_1) + b_1c_1)Y$$



$GF(2^4)$  在基  $(Y^4, Y)$  上的乘法:

$$Z^2Y^4(b_0Z^2Y^4 + b_1ZY^4 + b_2Z^2Y + b_3ZY)^2 = b_0Z^2Y^4 + (b_0 + b_1)ZY^4 + (b_1 + b_3)Z^2Y + (b_0 + b_2)ZY$$

$GF(2^8)$  在基  $(X^{16}, X)$  上的求逆:

$$(b_0X^{16} + b_1X)^{-1} = (b_0b_1 + Z^2Y^4(b_0 + b_1)^2)^{-1}(b_1X^{16} + b_0X)$$

算法 7  $S_1$  构造算法 (见图 6)

1.  $a = M_2 \times (x \oplus \text{Con}_2)$ , 将  $a$  分为 2 个 4 比特  $a_0, a_1$ , 即  $a = a_0 \| a_1$
2.  $b_0 = a_0 \oplus a_1, b_1 = a_0 \otimes a_1$
3.  $c = Z^2Y^4 \otimes b_0^2$
4.  $d = c \oplus b_1$
5.  $e = d^{-1}$
6.  $f_0 = e \otimes a_1, f_1 = e \otimes a_0$
7.  $S_1[x] = M_3 \times (f_0 \| f_1) \oplus \text{Con}_3$

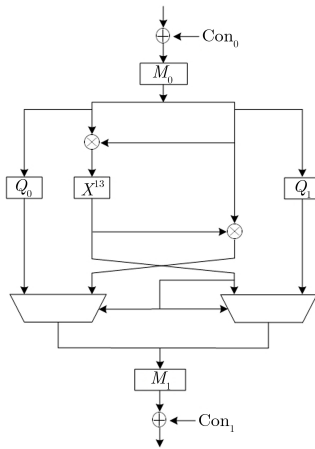


图 5  $S_0$  结构图

Figure 5 Construction of  $S_0$

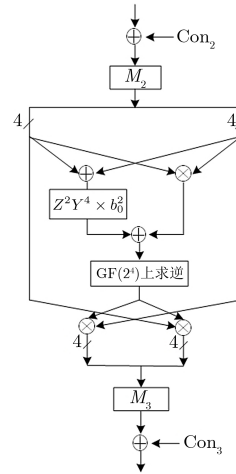


图 6  $S_1$  结构图

Figure 6 Construction of  $S_1$

设  $S_1$  的 8 比特输入为  $x$ , 则如算法 7 计算  $S_1$  的 8 比特输出  $S_1[x]$ .

$\otimes$  表示  $GF(2^4)$  在基  $(Z^2Y^4, ZY^4, Z^2Y, ZY)$  上的乘法.  $d^{-1}$  表示  $GF(2^4)$  在相同基上  $d$  的逆, 该变换用 16 进制表示为:

$$0x0, 0x4, 0xc, 0x8, 0x1, 0xa, 0xe, 0xd, 0x3, 0xb, 0x5, 0x9, 0x2, 0x7, 0x6, 0xf$$

两个矩阵  $M_2$  和  $M_3$  都是置换矩阵,  $M_2$  的第  $i$  行第  $\tau_2(i)$  列为 1, 其余为 0;  $M_3$  的第  $i$  行第  $\tau_3(i)$  列为 1, 其余为 0;  $0 \leq i \leq 7$ .  $\tau_2$  和  $\tau_3$  分别为:

$$4, 6, 5, 1, 0, 7, 2, 3$$

$$5, 1, 3, 6, 2, 4, 0, 7$$

两个 8 比特常数分别为:  $\text{Con}_2 = 0xa4$ ,  $\text{Con}_3 = 0x5f$ .

## 2 设计原理

### 2.1 加解密过程设计原理

整体结构采用标准 Feistel 结构, 其优点在于:

- (1) 加解密过程除子密钥的顺序外完全相同, 降低软硬件的实现代价.
- (2) 与 SPN 结构相比, 其使用的硬件资源更小. 更易做资源与效率的折中.

轮函数使用 SP 结构, 其主要考虑如下:

- (1) 安全性和有效性的综合性能更高.
- (2) 基本运算为查表、固定位数的循环移位, 以及异或等逻辑运算, 易于硬件实现以及软件的查大表快速实现.

通过整体结构以及轮函数的设计, SMBA 算法具备软硬件多种平台上实现的有效性和灵活性. SMBA-128 与 SMBA-256 共用主要的密码部件, 一起实现所付出的额外代价低. 同时, 算法可以准确评估抵抗差分攻击、线性攻击、代数攻击和积分攻击等已知攻击所需的轮数, 安全界清晰. 所选迭代轮数确保算法有较大的安全冗余. 在算法设计上不隐藏任何“后门”.

## 2.2 密钥扩展设计原理

- (1) 一次加密子密钥、解密子密钥的生成速度分别高于一个分组加密、解密运算的速度. 这样可以尽量降低在短消息条件下密钥扩展过程对算法性能的影响.
- (2) 加密子密钥和解密子密钥均可在加解密过程中实时计算, 且与加解密过程相比硬件实现电路深度更小, 可以为硬件实现提供更多的灵活性、更好的有效性.

综合以上的考虑, 我们设计了 SMBA 的密钥扩展算法. SMBA-128-128 和 SMBA-256-256 密钥扩展的结构本质上是广义 Feistel 结构<sup>[1]</sup>, SMBA-128-256 密钥扩展是广义 Feistel 结构的推广. 此外密钥扩展与加解密过程共用 S 盒  $S_0$  和  $S_1$ , 在特定场景下可通过复用的方式降低硬件资源.

## 2.3 S 盒设计原理

SMBA 使用 8 比特 S 盒, 主要的考虑是 8 比特 S 盒逻辑更加复杂, 相比较 4 比特 S 盒, 其抵抗潜在的攻击能力更强. 在非轻量级环境下, 一般这样的 S 盒的实现代价是可以接受的.

SMBA 所用的 8 比特 S 盒  $S_0$  和  $S_1$  通过两种不同的方式构造生成 (设计思想分别参考了文献 [2,3] 和文献 [4]), 其用到的非线性运算为查 4 比特 S 盒,  $GF(2^4)$  上的乘法, 以及多路选择器. 相比于随机产生然后进行检测生成的 S 盒 (还包括梯度下降、遗传算法等的迭代优化),  $S_0$  和  $S_1$  有如下优势:

- (1) 更好的密码强度指标.
- (2) 更优的硬件实现性能. 在 4 比特 S 盒的选取上, 我们尽量采用硬件资源小的 S 盒, 同时通过选取特定的基, 使得  $GF(2^4)$  上的乘法运算可以节省一定的资源, 可以追求更小的面积. 我们也有追求更小深度的实现方式, 更适用于追求更高速度的环境.
- (3) 更低的侧信道攻击防护代价. 在 4 比特 S 盒的选取上, 我们除了实现资源外, 更考虑了其中与门的个数. 通过采用具有更少与门的 S 盒, 并配以搜索得到的前后仿射变换, 我们获得的 S 盒在抵抗侧信道攻击的指标更优.
- (4) 可清楚地表明  $S_0$  和  $S_1$  的设计均无后门.

## 2.4 线性变换和换位变换设计原理

由于 64 比特的 MDS 线性变换软硬件实现代价较大, 关键路径深, 资源占用多, 我们希望在密码学性质和实现代价之间取得折中. SMBA 线性变换的主要设计思想是将少量简单的线性变换组合起来, 获得具有更高分支数、更好密码学性质的大规模线性变换. 我们采用了两个简单的  $L_{32}$  变换, 以及一个 L-M 结构的简单线性变换, 成功构造了一个新的变换  $L_{64}$ , 其具有如下的特性:

- (1)  $L_{64}$ 、 $L_{64}^T$  的差分分支数和线性分支数都为 6.
- (2)  $L_{64}$ 、 $L_{64}^T$  都将 1 个非零字节变换到 7 个非零字节.
- (3) 不是面向字节的运算.
- (4) 各种软件平台 (8/32/64 比特) 及硬件都可有效实现.

对 SMBA-256, 换位变换  $P_{128}$  不仅确保算法的加解密过程都形成一个整体, 并且使得算法抗差分、线性等攻击能力显著提升.

3 安全性分析

3.1 整体结构安全性

SMBA 的整体结构采用标准 Feistel 结构, Feistel 结构已广泛应用于分组密码算法的设计中, 具有很好的结构安全性, 可证明: 在轮函数是伪随机函数的假设下, 3 轮 Feistel 结构是伪随机置换, 4 轮 Feistel 结构是超伪随机置换.

另外, 对于 SMBA-256, 若假设函数  $L_{64} \circ S_{64}$  为伪随机函数 (相对于轮函数是伪随机函数的假设是更弱、更合理的假设), 则可证明: 4 轮是伪随机置换, 6 轮是超伪随机置换.

3.2 密码部件安全性

3.2.1 非线性部件安全性

SMBA 算法使用了 2 个 8 比特到 8 比特的 S 盒  $S_0$  和  $S_1$ , 其密码学性质如表 2.

表 2  $S_0$  和  $S_1$  的密码学性质  
Table 2 Cryptographic property of  $S_0$  and  $S_1$

| 对比项                              | $S_0$                     | $S_1$          |
|----------------------------------|---------------------------|----------------|
| 不动点                              | 无                         | 无              |
| 平衡性                              | 平衡                        | 平衡             |
| 完全性                              | 完全                        | 完全             |
| 代数次数                             | 7                         | 7              |
| 最大差分概率                           | $6/256 = 2^{-5.415}$      | $2^{-6}$       |
| 最大线性概率                           | $(20/128)^2 = 2^{-5.356}$ | $2^{-6}$       |
| 非线性度                             | 108                       | 112            |
| 二次关系                             | 不存在                       | 39 个线性无关的二次关系  |
| 三次关系                             | 441 个线性无关的三次关系            | 471 个线性无关的三次关系 |
| 透明阶 <sup>[2]</sup>               | 7.770                     | 7.857          |
| 改进的透明阶 <sup>[2]</sup>            | 6.852                     | 6.940          |
| DPA 信噪比 <sup>[2]</sup>           | 7.470                     | 9.059          |
| Hamming 重量的混乱系数方差 <sup>[2]</sup> | 0.225                     | 0.133          |

其中, 透明阶、改进的透明阶、DPA 信噪比、Hamming 重量的混乱系数方差是为了评估 S 盒抵抗侧信道攻击 (SCA) 的能力引入的指标. 通常认为, 透明阶、改进的透明阶和 DPA 信噪比越小, 表明 S 盒抵抗差分能量攻击 (DPA) 的能力越强; Hamming 重量的混乱系数方差越大, 表明 S 盒抵抗侧信道攻击的能力越强. AES 算法 S 盒的透明阶、改进的透明阶、DPA 信噪比、Hamming 重量的混乱系数方差分别为 7.860、6.916、9.600、0.111, 根据表 2 知,  $S_0$  和  $S_1$  比 AES 算法的 S 盒具有更强的抗侧信道攻击的能力 (唯一的例外是 AES 算法 S 盒改进的透明阶小于  $S_1$  改进的透明阶).

3.2.2 线性部件安全性

SMBA 的线性变换有  $L_{32}$ 、 $L_{64}$ 、 $L_{128}$  三种.

$L_{32}$  具有以下性质:

- (1)  $L_{32}$  是对合变换, 即  $L_{32}(L_{32}(x)) = x$ ;
- (2)  $L_{32}$  的差分分支数和线性分支数都是 4.

$L_{64}$  具有以下性质:

- (1)  $L_{64}$  的差分分支数和线性分支数都是 6;
- (2)  $y = L_{64}(x)$ ,  $w(x) = 1$  时,  $w(y) \geq 7$ ;
- (3)  $y = L_{64}^T(x)$ ,  $w(x) = 1$  时,  $w(y) \geq 7$ ;
- (4)  $(y_0 || y_1) = L_{64}(x_0 || x_1)$ , 若  $x_0 = 0$ ,  $x_1 \neq 0$ , 则  $y_0 \neq 0$ , 且  $y_1 = 0 \Leftrightarrow x_1 = 0\text{xfffffff}$ ;

(5)  $(y_0||y_1) = L_{64}(x_0||x_1)$ , 若  $x_0 \neq 0, x_1 = 0$ , 则  $y_1 \neq 0$ , 且  $y_0 = 0 \Leftrightarrow x_0 = 0\text{xffffffff}$ ;  
 $L_{128}$  分支数等指标与  $L_{64}$  一致.

3.3 抗攻击安全

3.3.1 差分攻击

采用截断差分的思想分析 SMBA 算法抵抗差分攻击的能力. Mouha<sup>[5]</sup>给出了一种采用混合整数规划模型搜索差分活动 S 盒的方法, 我们结合  $S_0$  和  $S_1$  的差分概率权重, 分别评估 SMBA-128 和 SMBA-256 具有的最大差分特征概率. SMBA-128 经过 13 轮迭代后最大差分特征概率不大于  $2^{-129.96}$ , 小于安全界  $2^{-128}$ ; SMBA-256 经过 18 轮迭代后最大差分特征概率不大于  $2^{-264.600}$ , 小于安全界  $2^{-256}$ . 最大差分特征概率搜索结果详见表 3、表 4.

表 3 SMBA-128 差分概率  
Table 3 Differential probability of SMBA-128

| 轮数 | 差分特征概率上界      | 轮数 | 差分特征概率上界       | 轮数 | 差分特征概率上界       | 轮数 | 差分特征概率上界       |
|----|---------------|----|----------------|----|----------------|----|----------------|
| 1  | 0             | 7  | $2^{-61.320}$  | 13 | $2^{-129.960}$ | 19 | $2^{-194.940}$ |
| 2  | $2^{-5.415}$  | 8  | $2^{-75.810}$  | 14 | $2^{-140.790}$ | 20 | $2^{-205.770}$ |
| 3  | $2^{-10.830}$ | 9  | $2^{-86.640}$  | 15 | $2^{-151.620}$ | 21 | $2^{-216.600}$ |
| 4  | $2^{-32.490}$ | 10 | $2^{-97.470}$  | 16 | $2^{-162.450}$ | 22 | $2^{-227.430}$ |
| 5  | $2^{-43.320}$ | 11 | $2^{-108.300}$ | 17 | $2^{-173.280}$ | 23 | $2^{-238.260}$ |
| 6  | $2^{-54.150}$ | 12 | $2^{-119.130}$ | 18 | $2^{-184.110}$ | 24 | $2^{-249.090}$ |

表 4 SMBA-256 差分概率  
Table 4 Differential probability of SMBA-256

| 轮数 | 差分特征概率上界      | 轮数 | 差分特征概率上界       | 轮数 | 差分特征概率上界       | 轮数 | 差分特征概率上界       |
|----|---------------|----|----------------|----|----------------|----|----------------|
| 1  | 0             | 7  | $2^{-87.810}$  | 13 | $2^{-184.110}$ | 19 | $2^{-276.600}$ |
| 2  | $2^{-5.415}$  | 8  | $2^{-105.225}$ | 14 | $2^{-194.940}$ | 20 | $2^{-288.600}$ |
| 3  | $2^{-10.830}$ | 9  | $2^{-116.640}$ | 15 | $2^{-210.450}$ | 21 | $2^{-309.090}$ |
| 4  | $2^{-38.490}$ | 10 | $2^{-137.715}$ | 16 | $2^{-232.110}$ | 22 | $2^{-321.090}$ |
| 5  | $2^{-55.320}$ | 11 | $2^{-149.130}$ | 17 | $2^{-244.110}$ | 23 | $2^{-333.090}$ |
| 6  | $2^{-76.395}$ | 12 | $2^{-170.790}$ | 18 | $2^{-264.600}$ | 24 | $2^{-354.165}$ |

3.3.2 线性攻击

与差分攻击方法相似可得, SMBA-128 经过 13 轮迭代后最大线性特征概率不大于  $2^{-128.544}$ , 小于安全界  $2^{-128}$ ; SMBA-256 经过 18 轮迭代后最大线性特征概率不大于  $2^{-262.240}$ , 小于安全界  $2^{-256}$ . 最大线性特征概率搜索结果详见表 5、表 6.

3.3.3 代数攻击

SMBA 算法的  $S_0$  不存在二次关系,  $S_1$  存在 39 个二次关系, 可构造的二次方程数量低于 AES, 涉及的变量数不低于 AES, 使用 XL 算法求解构造的二次方程组求解复杂度应高于 AES, 可认为 SMBA 能够抵抗代数攻击.

3.3.4 积分攻击

对于 128 比特分组的 SMBA-128 算法, 令算法输入的多重集合具有可除性  $D_K^{8,16}$ , 则经 7 轮加密后输出的多重集合的第 9 到第 16 个字节为平衡字节, 而经 8 轮加密后输出的多重集合的每个字节都是不确定字节, 即算法存在 7 轮积分区分器. 对于 256 比特分组的 SMBA-256 算法, 令算法输入的多重集合具有可除性  $D_K^{8,32}$ , 则经 8 轮加密后输出的多重集合的第 17 到第 32 个字节为平衡字节, 而经 9 轮加密后输出的多重集合的每个字节都是不确定字节, 即算法存在 8 轮积分区分器.



表 8 SMBA-128-256 相关密钥差分概率  
Table 8 Related-key differential probability of SMBA-128-256

| 轮数 | 差分特征概率上界      | 轮数 | 差分特征概率上界       | 轮数 | 差分特征概率上界       | 轮数 | 差分特征概率上界       |
|----|---------------|----|----------------|----|----------------|----|----------------|
| 1  | 0             | 7  | $2^{-54.735}$  | 13 | $2^{-134.055}$ | 19 | $2^{-233.865}$ |
| 2  | $2^{-5.415}$  | 8  | $2^{-66.150}$  | 14 | $2^{-148.545}$ | 20 | $2^{-248.205}$ |
| 3  | $2^{-5.415}$  | 9  | $2^{-72.735}$  | 15 | $2^{-165.375}$ | 21 | $2^{-259.620}$ |
| 4  | $2^{-10.830}$ | 10 | $2^{-84.150}$  | 16 | $2^{-176.790}$ | 22 | $2^{-281.865}$ |
| 5  | $2^{-16.245}$ | 11 | $2^{-106.980}$ | 17 | $2^{-198.450}$ | 23 | $2^{-294.450}$ |
| 6  | $2^{-39.075}$ | 12 | $2^{-122.640}$ | 18 | $2^{-213.960}$ | 24 | $2^{-324.600}$ |

表 9 SMBA-256-256 相关密钥差分概率  
Table 9 Related-key differential probability of SMBA-256-256

| 轮数 | 差分特征概率上界      | 轮数 | 差分特征概率上界      | 轮数 | 差分特征概率上界       | 轮数 | 差分特征概率上界       |
|----|---------------|----|---------------|----|----------------|----|----------------|
| 1  | 0             | 4  | $2^{-32.490}$ | 7  | $2^{-103.320}$ | 10 | $2^{-201.810}$ |
| 2  | $2^{-5.415}$  | 5  | $2^{-55.320}$ | 8  | $2^{-126.735}$ | 11 | $2^{-251.865}$ |
| 3  | $2^{-16.245}$ | 6  | $2^{-69.075}$ | 9  | $2^{-156.735}$ | 12 | $2^{-304.110}$ |

3.3.8 滑动攻击

由于 SMBA 密钥扩展算法的设计, 可证明 SMBA-128-128 和 SMBA-256-256 对相同密钥或是两个不同密钥, 不会有任意连续 3 轮子密钥均相同; 对 SMBA-128-256, 对相同密钥或是两个不同密钥, 不会有连续 5 轮子密钥相同. 因此我们认为 SMBA 可以抵抗滑动攻击.

3.3.9 密钥扩展安全性

SMBA 的密钥扩展算法具有保熵性. 通过仔细的分析, 我们认为 SMBA 不具有等价密钥、弱密钥或半弱密钥.

3.4 侧信道攻击安全防护

SMBA 算法的 2 个 S 盒都采用代数构造的方式设计, 具有较好的透明阶、改进的透明阶、Hamming 重量的混乱系数方差等指标, 使得在相同安全防护条件下, 抵抗侧信道攻击的能力更强. 另外, S 盒掩码防护实现难度 (芯片面积占用、随机数使用量等) 和 S 盒逻辑表达式的乘法复杂度密切相关, 逻辑复杂度越低, 安全防护实现时使用的随机数量越少、芯片占用面积越低, 因此, 在  $S_0$  和  $S_1$  构造过程中本算法尽量使用乘法复杂度低的部件、运算, 使得  $S_0$  和  $S_1$  的乘法复杂度低.

4 性能分析

4.1 软件性能

算法的软件性能测试标准是在 CBC 模式下, 用 256 字节数据作为输入进行性能测试, 每次测试变换密钥, 取  $10^5$  次 CBC 模式运算测试结果的平均值作为速率测试的结果.

32 比特 ARM 环境采用基于 STM32 F103 的开发板 (主频 72 Mhz): stm32f1zet6 核心板 6、512 K 片内 Flash、64 K 片内 RAM. 算法采用查表方式实现, 实现结果见表 10.

表 10 32 比特 ARM 环境下 SMBA 算法性能  
Table 10 Performance of SMBA on 32-bit ARM platform

| 算法           | 加密速率 (Mbps) | 解密速率 (Mbps) |
|--------------|-------------|-------------|
| SMBA-128-128 | 6.84        | 6.71        |
| SMBA-128-256 | 3.48        | 3.48        |
| SMBA-256-256 | 3.91        | 3.90        |

SMBA 的 64 位软件实现平台采用 Intel Core(TM) i7-9750H CPU、主频 2.6 GHz、Windows10、内存 32 G、Microsoft Visual Studio 2017 编译器. 算法加密过程采用查表方式实现, 解密过程使用 AVX 指令实现, 实现结果见表 11.

表 11 64 比特 Windows 环境下 SMBA 算法性能  
Table 11 Performance of SMBA on 64-bit Windows platform

| 算法           | 加密速率 (Mbps) | 解密速率 (Mbps) |
|--------------|-------------|-------------|
| SMBA-128-128 | 1405        | 2751        |
| SMBA-128-256 | 1110        | 2035        |
| SMBA-256-256 | 1655        | 2831        |

4.2 硬件性能

算法的 ASIC 实现环境, 采用的设计库为 HJTC0.11 um 工艺库, 采用的综合工具为 Synopsys Design Vision (F-2011.09-SP1 Version). 仿真环境为 Modelsim SE 10.1a. 算法测试 32 个分组长度数据, 测试结果见表 12.

表 12 ASIC 环境下 SMBA 算法性能  
Table 12 Performance of SMBA on ASIC

| 对比项   | SMBA-128-128           | SMBA-128-256           | SMBA-256-256            |
|-------|------------------------|------------------------|-------------------------|
| 运算周期  | 627                    | 825                    | 825                     |
| 时钟约束  | 11 ns                  | 11 ns                  | 11 ns                   |
| 加密速率  | 566.37 Mbps            | 430.44 Mbps            | 860.88 Mbps             |
| 解密速率  | 566.37 Mbps            | 430.44 Mbps            | 860.88 Mbps             |
| 面积    | 69 065 um <sup>2</sup> | 87 851 um <sup>2</sup> | 164 683 um <sup>2</sup> |
| 加密吞面比 | 0.008 20               | 0.004 90               | 0.005 23                |
| 解密吞面比 | 0.008 20               | 0.004 90               | 0.005 23                |

5 优缺点分析

SMBA 是安全性高、实用性强、创新设计的分组密码算法, 算法的设计具备简洁性、灵活性和兼容性, 完全能够满足本次算法设计竞赛的要求.

5.1 算法特色

- (1) 算法设计简洁, 三个版本共用非线性变换  $S_0$  和  $S_1$ , 以及线性变换  $L_{64}$ , 扩展自然.
- (2) 算法适用于多种软件平台, 如 8/32/64 位平台, 且算法的软件性能高, 实用性强.
- (3) 算法安全性分析充分, 安全界清晰, 具有较高的安全冗余.
- (4) 算法具有良好的抵抗侧信道攻击的能力. 由于我们在  $S_0$  和  $S_1$  的设计中充分考虑到了侧信道防护的问题,  $S_0$  和  $S_1$  在透明阶、DPA 信噪比、Hamming 重量混乱系数方差等评价侧信道防护能力的指标上都优于 AES 的 S 盒. 在  $S_0$  和  $S_1$  构造过程中本算法尽量使用乘法复杂度低的部件、运算, 使得  $S_0$  和  $S_1$  的乘法复杂度低. 由于针对线性变换的侧信道防护策略是平凡的, 所以我们认为 SMBA 具有良好的抗侧信道攻击能力.

5.2 创新点

- (1) 创新设计了密码性质优、软硬件实现性能高的线性变换  $L_{64}$ : 与同等规模的 MDS 变换相比,  $L_{64}$  的实现代价非常小, 其只由两个简单的  $L_{32}$  变换和一个 L-M 结构线性变换构成, 软硬件实现简单, 但同时其分支数可达到 6, 且能够将 1 个非零字节变换为 7 个非零字节, 具有良好的密码学

性质. 利用  $L_{64}$  的密码性质, 通过理论证明和计算机搜索 (两者结果相同) 均可证明, SMBA-128 具有很高的抵抗差分攻击和线性攻击的能力.

- (2) SMBA-256 整体设计保证安全: 换位变换  $P_{128}$  结合两个并置的线性变换  $L_{64}$ , 确保 SMBA-256 的加解密过程都形成一个整体, 大幅度提升密码算法的扩散性和抗攻击能力.
- (3) 面向实现优化设计: 设计了密码指标高的 S 盒  $S_0$  和  $S_1$ , 尤其是  $S_0$  的设计. 与已有主要密码指标相同的 S 盒相比, 硬件实现电路的门数和深度均有减少.

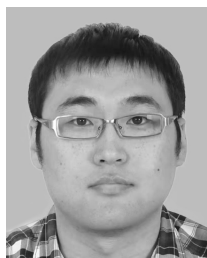
### 5.3 不足

由于设计中采用了 8 比特 S 盒, 且分组长度/密钥长度大, 因此 SMBA 算法不适用于低功耗和硬件资源受限的轻量级应用环境.

## References

- [1] SUZUKI T, MINENATSU K. Improving the generalized Feistel[C]. In: Fast Software Encryption—FSE 2010. Springer Berlin Heidelberg, 2010: 19–39. [DOI: 10.1007/978-3-642-13858-4\_2]
- [2] ANTONIO R. On some methods for constructing almost optimal S-Boxes and their resilience against side-channel attacks[J]. IACR Cryptology ePrint Archive, 2018: 2018/618. <https://eprint.iacr.org/2018/618.pdf>
- [3] BIRYUKOV A, PERRIN L, UDOVENKO A. Reverse engineering the S-box of Streebog, Kuznyechik and STRIBOBr1[C]. In: Advances in Cryptology—EUROCRYPT 2016, Part I. Springer Berlin Heidelberg, 2016: 372–402. [DOI: 10.1007/978-3-662-49890-3\_15]
- [4] ABBASI I, AFZAL M. A compact S-box design for SMS4 block cipher[J]. IACR Cryptology ePrint Archive, 2011: 2011/522. <https://eprint.iacr.org/2011/522.pdf>
- [5] MOUHA N, WANG Q, GU D, et al. Differential and linear cryptanalysis using mixed-integer linear programming[C]. In: Information Security and Cryptology—INSCRYPT 2011. Springer Berlin Heidelberg, 2011: 57–76. [DOI: 10.1007/978-3-642-34704-7\_5]
- [6] WU S B, WANG M S. Automatic search of truncated impossible differentials for word-oriented block ciphers[C]. In: Progress in Cryptology—INDOCRYPT 2012. Springer Berlin Heidelberg, 2012: 283–302. [DOI: 10.1007/978-3-642-34931-7\_17]

## 作者信息



王克 (1985–), 辽宁盖州人, 硕士, 工程师. 主要研究领域为密码理论与应用.  
wangke3107@163.com



贾文义 (1982–), 山西吕梁人, 硕士, 高级工程师. 主要研究领域为密码理论与应用.  
jiawy163@163.com



黄念 (1989–), 福建莆田人, 硕士, 工程师. 主要研究领域为密码理论与应用.  
nnhuang205@163.com



## 附录

SMBA-128-128 测试向量:

密钥: 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00.

明文: 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00.

密文: 0x21,0x9a,0x3b,0x8f,0x61,0x47,0x98,0x64,0x8f,0xc4,0xd6,0xd1,0x9d,0x72,0x86,0x6d.

SMBA-128-256 测试向量:

密钥: 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,  
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00.

明文: 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00.

密文: 0x40,0x76,0x91,0x2d,0xfc,0x2b,0x9b,0xe2,0x20,0xb9,0x76,0x27,0xe1,0x72,0xf3,0x33.

SMBA-256-256 测试向量:

密钥: 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,  
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00.

明文: 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,  
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00.

密文: 0x1b,0x8b,0x33,0x77,0x1d,0x34,0xa3,0x64,0xd2,0x89,0x20,0x99,0xb3,0xd9,0x0b,0x1f,  
0x43,0x62,0x50,0xea,0x73,0x6c,0x45,0x30,0xa3,0xd3,0xf8,0xa4,0xab,0x1c,0xb7,0x59.